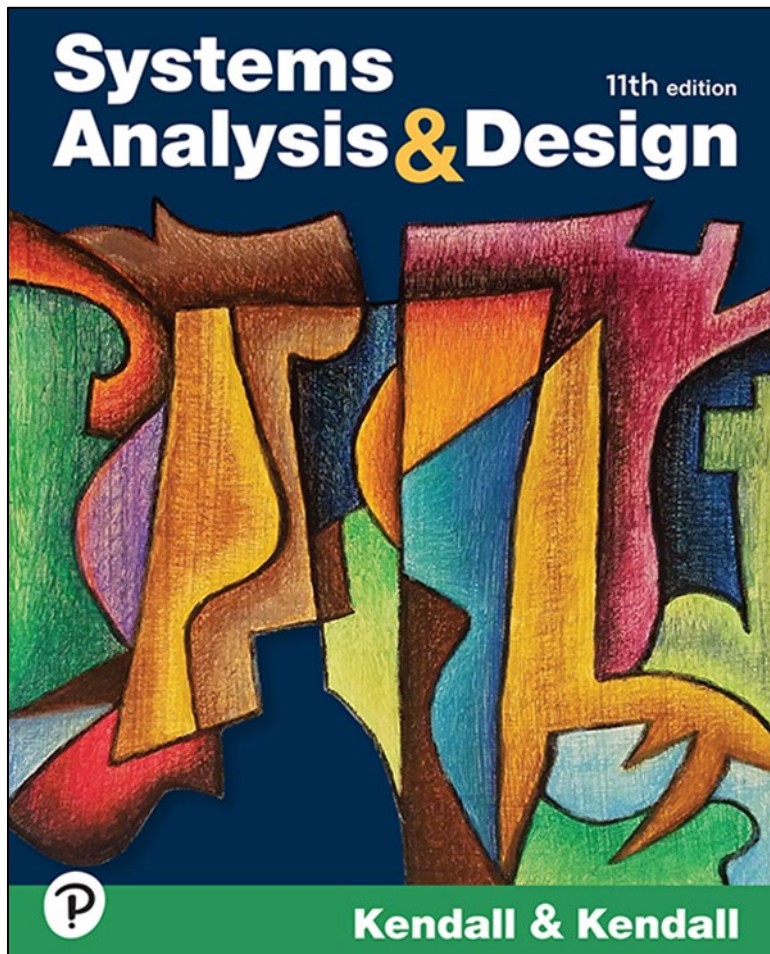


Systems Analysis & Design

Eleventh Edition



Chapter # 1

Systems, Roles, and
Development
Methodologies

Learning Objectives (1 of 2)

1.1 Understand the need for systems analysis and design in organizations.

1.2 Realize what the many roles of a system analyst are.

1.3 Comprehend the fundamentals of the systems development life cycle (SDLC).

1.4 Understand the agile methodology for systems development.

Learning Objectives (2 of 2)

1.5 Gain an appreciation for object-orientated systems design.

1.6 Understand the importance of cloud computing and the cloud development life cycle (CDLC).

1.7 Learn how to choose which systems development method to use.

1.8 Discover what open source software is and how it is developed.

Information – A Key Resource

- Fuels business and can be the critical factor in determining the success or failure of a business
- Needs to be managed correctly
- Managing computer-generated information differs from handling manually produced data

Major Topics

- Importance of systems analysis and design
- Roles of systems analysts
- Fundamentals of different kinds of information systems
- Phases in the systems development life cycle as they relate to Human-Computer Interaction (HCI) factors
- CDLC is introduced and contrasted to the SDLC
- Agile development
- Object-oriented Systems Analysis and Design

Need for Systems Analysis and Design

- Security is a critical factor and a challenge when developing new systems
- Installing a system without proper planning leads to great user dissatisfaction and frequently causes the system to fall into disuse
- Systems analysis and design lends structure to the analysis and design of information systems
- User involvement throughout a systems project is critical to the successful development of computerized systems

Roles of the Systems Analyst

The analyst must be able to work with people of all descriptions and be experienced in working with computers

- Three primary roles:
 - Consultant
 - Supporting expert
 - Agent of change

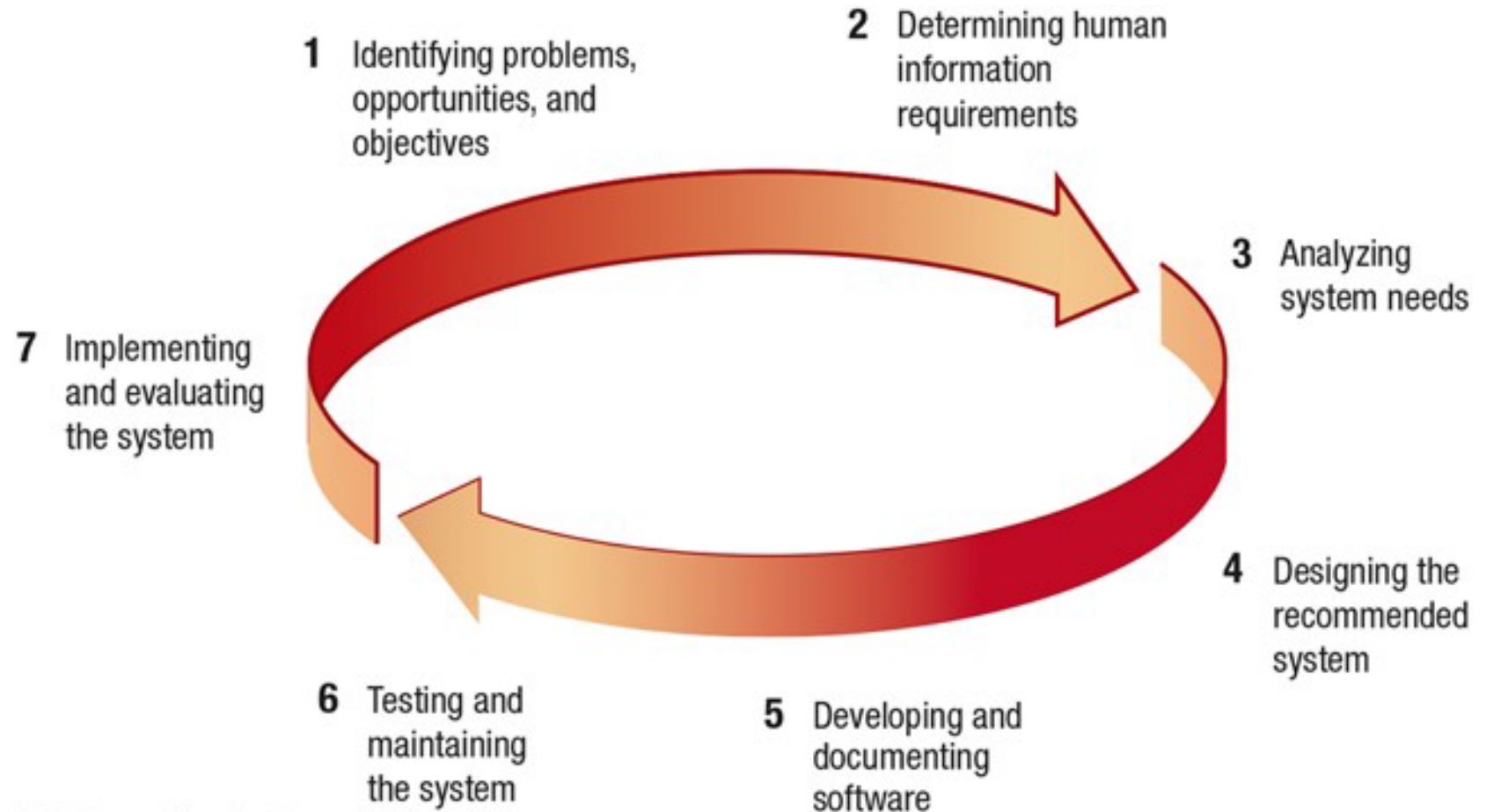
Qualities of a System Analyst

- Problem solver
- Communicator
- Strong personal and professional ethics
- Self-disciplined and self-motivated

Systems Development Life Cycle (SDLC)

- The systems development life cycle is a phased approach to analysis and design processes
- Developed through the use of a specific cycle of analyst and user activities
- Often referred to as the waterfall method

Figure 1.1 The Seven Phases of the Systems Development Life Cycle



Identifying Problems, Opportunities, and Objectives

- Activity:
 - Interviewing user management
 - Summarizing the knowledge obtained
 - Estimating the scope of the project
 - Documenting the results
- Output:
 - Feasibility report containing problem definition and objective summaries from which management can make a decision on whether to proceed with the proposed project

Determining Human Information Requirements (1 of 2)

- Activity:
 - Interviewing
 - Sampling and investigating hard data
 - Questionnaires
 - Observe the decision maker's behavior and environment
 - Prototyping
 - Learn the who, what, where, when, how, and why of the current system

Determining Human Information Requirements (2 of 2)

- Output:
 - Analyst understands how users accomplish their work when interacting with a computer
 - Begin to know how to make the new system more useful and usable
 - The analyst should also know the business functions
 - Have complete information on the people, goals, data and procedure involved

Analyzing System Needs

- Activity:
 - Create data flow diagrams
 - Complete the data dictionary
 - Analyze the structured decisions made
 - Prepare and present the system proposal
- Output:
 - Recommendation on what, if anything, should be done

Designing the Recommended System

- Activity:
 - Design procedures to accurately enter data
 - Design the human–computer interface (HCI)
 - Design files and/or database
 - Design backup procedures
- Output
 - Model of the actual system

Developing and Documenting Software

- Activity:
 - System analyst works with coders to develop any original software needed
 - Works with users to develop effective documentation
 - Coder's design, code, and remove syntactical errors from computer programs
 - Document software with procedure manuals, online help, Frequently Asked Questions (FAQ) and Read Me files
- Output:
 - Computer programs
 - System documentation

Testing and Maintaining the System

- Activity:
 - Test the information system
 - System maintenance
 - Maintenance documentation
- Output:
 - Problems, if any
 - Updated programs
 - Documentation

Implementing and Evaluating the System

- Activity:
 - Train users
 - Plans the conversion from old system to new system
 - Review and evaluate system
- Output:
 - Trained personnel
 - Installed system

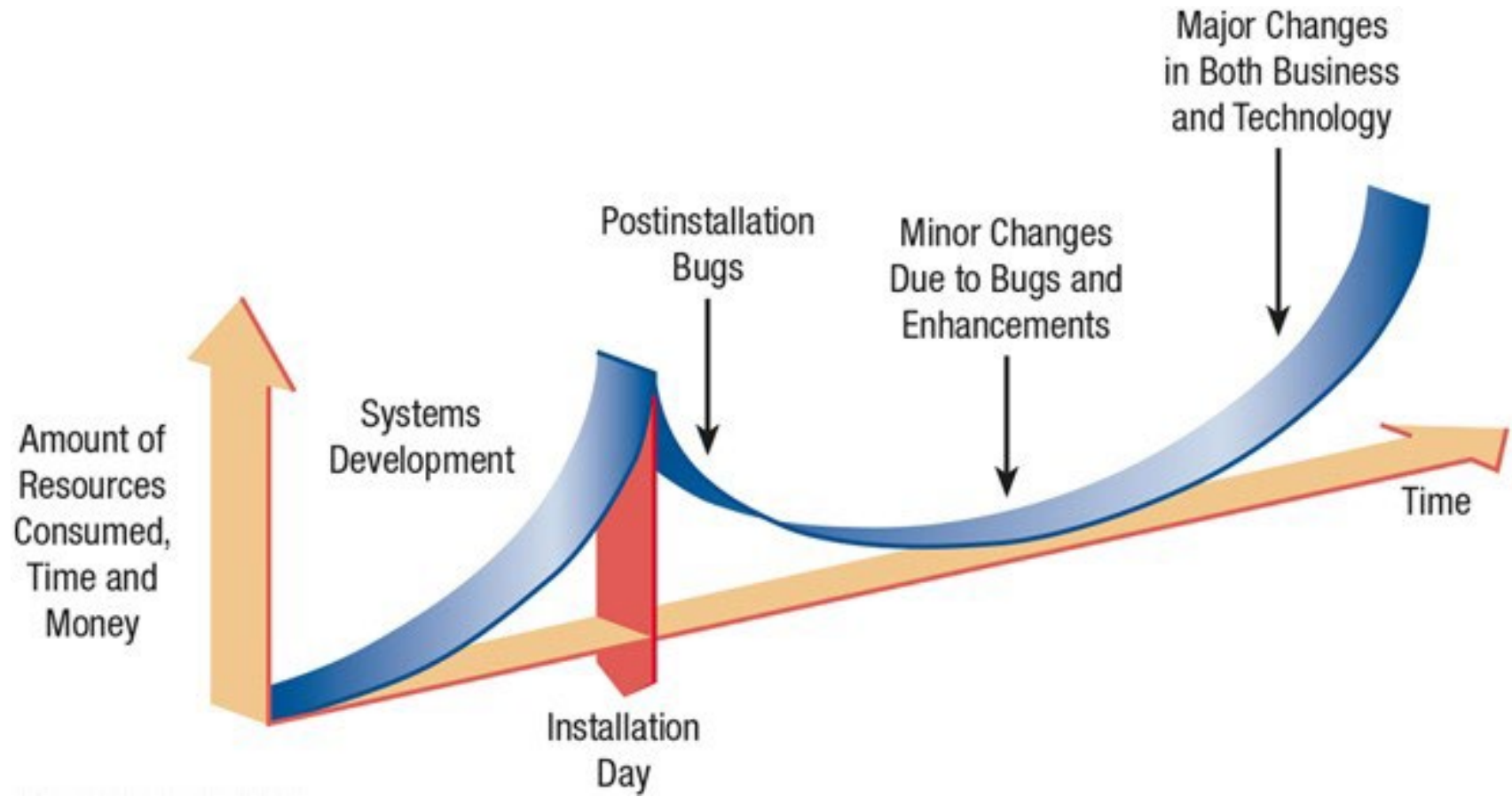
The Impact of Maintenance

- Maintenance is performed for two reasons
 - Correct software errors, and
 - Enhance software's capabilities in response to changing needs
- Enhance software for three reasons
 - Include additional features
 - Address business changes over time
 - Address hardware and software changes

Maintenance Impact

- Over time the cost of continued maintenance will be greater than that of creating an entirely new system
- At that point it becomes more feasible to perform a new systems study

Figure 1.2 Resource Consumption Over the System Life



Case Tools

- CASE (computer-aided software engineering) tools are productivity tools for systems analysts that have been created explicitly to improve their routine work through the use of automated support
- Reasons for using CASE tools:
 - Improving Analyst–User Communication
 - Help support modeling functional requirements
 - Assist in drawing project boundaries

The Agile Approach

- The agile approach is a software development approach based on:
 - Values
 - Principles
 - Core practices

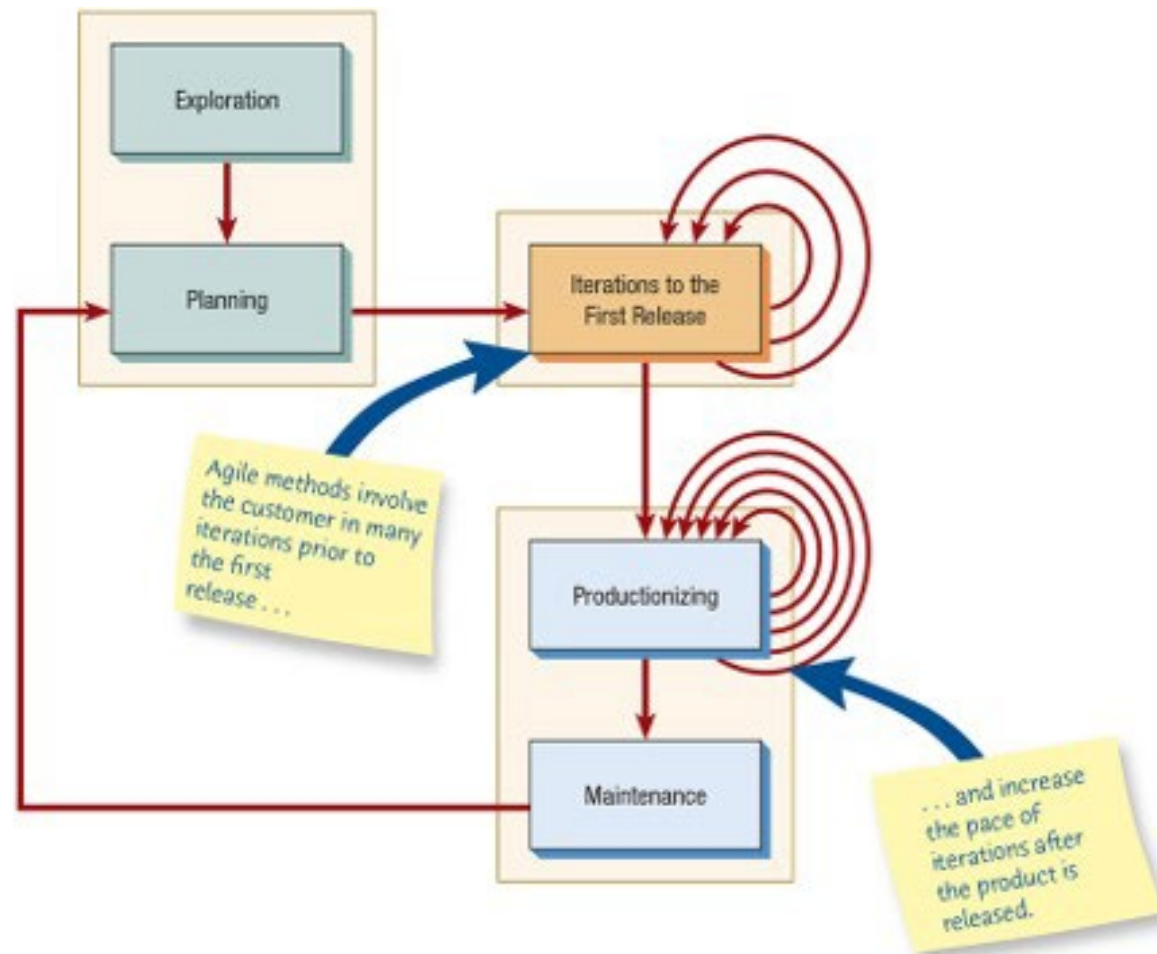
Agile Values

- The four values are:
 - Communication
 - Simplicity
 - Feedback
 - Courage

Agile Approach

- Agile approach is:
 - Interactive
 - Incremental
- Frequent iterations are essential for successful system development

Figure 1.4 The Five Stages of the Agile Modeling Development Process



Exploration

- Assemble team
- Assess skills
- Examine potential technologies
- Experiment with writing user stories
- Adopt a playful and curious attitude toward the work environment, its problems, technologies, and people

Planning

- Planning game - rules that can help formulate the agile development team's relationship with their business customers
- Maximize the value of the system produced by the agile team
- Main players are the development team and the business customer

Iterations to the First Release

- Iterations are cycles of:
 - Testing
 - Feedback
 - Change
- One goal is to run customer-written function tests at the end of each iteration

Productionizing

- The product is released in this phase
- May be improved by adding other features

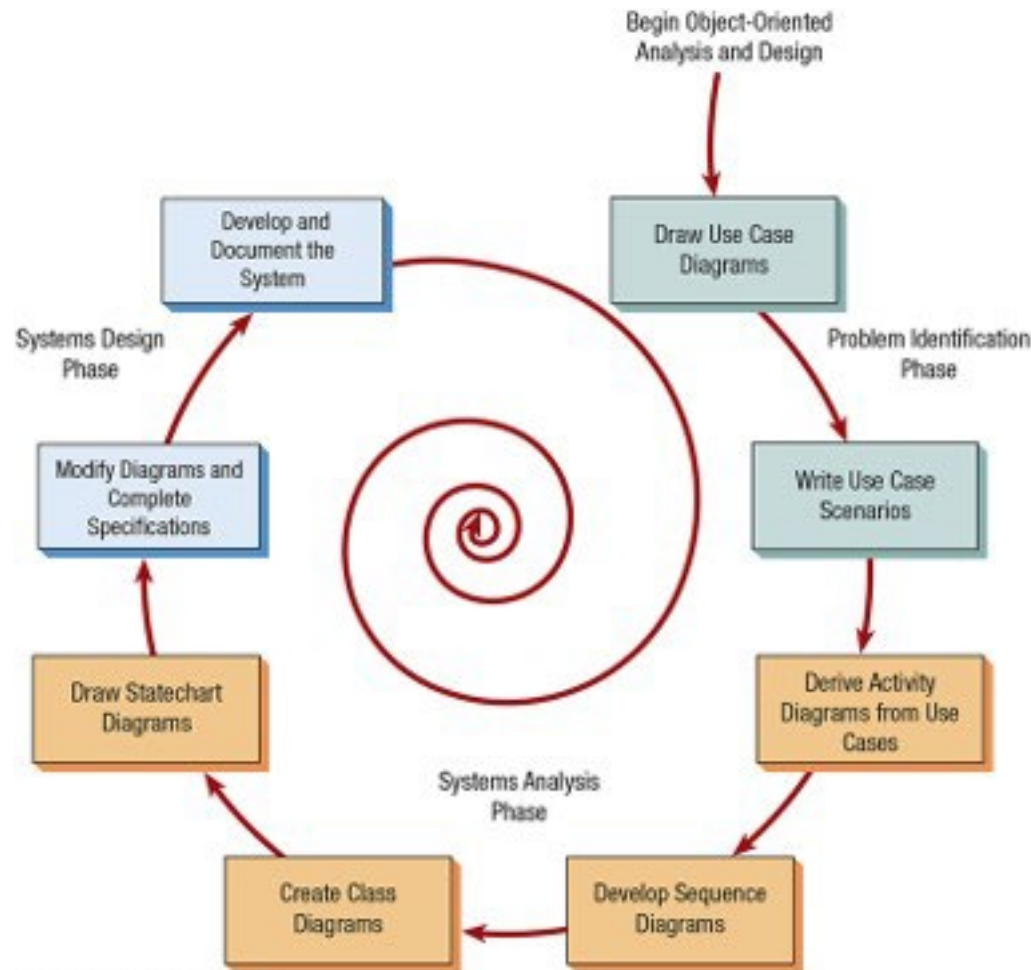
Maintenance

- New features may be added
- Riskier customer suggestions may be considered
- Team members may be rotated on or off the team

Object-Oriented Systems Analysis and Design

- Alternate approach to the structured approach of the SDLC that is intended to facilitate the development of systems that must change rapidly in response to dynamic business environments
- Use unified modeling language (UML) to model object-oriented systems
- Each object is a computer representation of some actual thing or event

Figure 1.5 The Steps in the UML Development Process



Problem Identification Phase

- Identify the actors and the major events initiated by the actors
- Draw a use case diagram - a diagram with stick figures representing the actors and arrows showing how the actors relate
- Write up a case scenario which describes in words the steps normally performed

Analysis Phase (1 of 2)

- Begin drawing UML diagrams
 - Draw activity diagrams, which illustrate all the major activities in the use case
 - Create one or more sequence diagrams for each use case that show the sequence of activities and their timing
 - Review the use cases, rethink them, and modify them if necessary

Analysis Phase (2 of 2)

- Develop class diagrams
- Draw statechart diagram

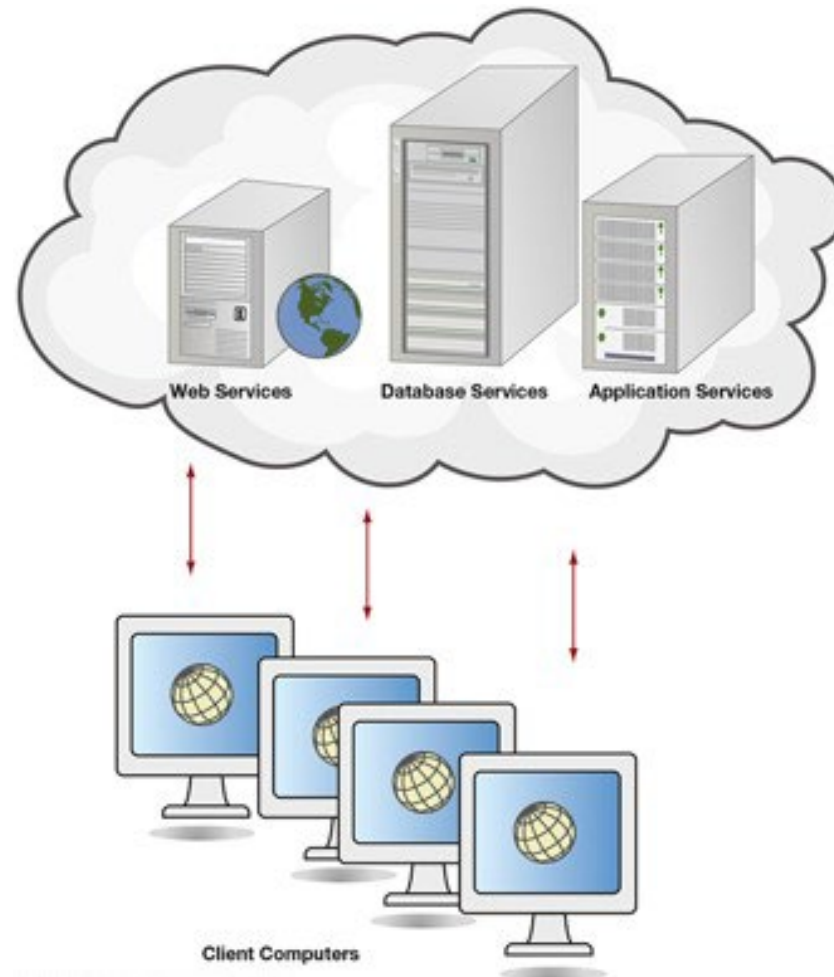
Systems Design

- Modify the UML diagrams drawn in the previous phase
 - Write class specifications for each class
- Develop and document the system
 - The more complete the information you provide to the development team through documentation and UML diagrams, the faster the development and the more solid the final production system

Cloud Computing

- Uses web services, database services, and applications over the internet without the investment of corporate or personal hardware, software, or software tools
- These “virtualized resources” have the ability to grow and adapt to changing business needs making them scalable to suit growing or changing demand by users
- Shared IT resources allow for cost sharing
- Improved disaster recovery due to redundant sites

Figure 1.6 Cloud Computing Offers Many Services



Cloud Computing Trade-Offs

- Concern about privacy and security are top concerns by organizations adopting cloud computing services
- Type of cloud service
 - Private
 - Public
 - Hybrid

Factors in Choosing a Cloud Provider

- Hacking
- Deletion of data
- Ease of switching providers
- Transferring data
- Privacy of data

ERP Systems and the Cloud

- Examples of ERP systems available in the cloud
 - Workday
 - NetSuite

Implementing CDLC (1 of 3)

- Cloud model five essential characteristics
 - On-demand self service
 - Broad network access
 - Resource pooling
 - Rapid elasticity
 - Measured service

Implementing CDLC (2 of 3)

- Three service models include
 - Software as a Service (SaaS)
 - Platform as a Service (Paas)
 - Infrastructure as a Service (IaaS)

Implementing CDLC (3 of 3)

- Four deployment models include
 - Private cloud
 - Community cloud
 - Public cloud
 - Hybrid cloud

Figure 1.7 How to Decide Which Development Method to Use

Choose	When
The Systems Development Life Cycle (SDLC) Approach	• systems have been developed and documented using SDLC • it is important to document each step of the way • upper-level management feels more comfortable or safe using SDLC • there are adequate resources and time to complete the full SDLC • communication of how new systems work is important
Agile Methodologies	• there is a project champion of agile methods in the organization • applications need to be developed quickly in response to a dynamic environment • a rescue takes place (the system failed and there is no time to figure out what went wrong) • the customer is satisfied with incremental improvements • executives and analysts agree with the principles of agile methodologies
Object-Oriented Methodologies	• the problems modeled lend themselves to classes • an organization supports the UML learning • systems can be added gradually, one subsystem at a time • reuse of previously written software is a possibility • it is acceptable to tackle the difficult problems first
The Cloud Development Life Cycle (CDLC) Approach	• the justification for the system project is to save money usually spent for on-premises data centers • the project can start small and build • multiple cloud providers can be used • changes created by the cloud can be monitored for visibility of apps and data from multiple systems • an integration strategy can be developed to capitalize on new synergies

Open Source Software (1 of 3)

- An alternative to traditional software development
- Many users and coders can study, share, and modify the code
- Program modifications must be shared with all people on the project and all licenses adhered to

Open Source Software (2 of 3)

- Categorized open source communities into four community types
 - Ad hoc
 - Standardized
 - Organized
 - Commercial

Open Source Software (3 of 3)

- Six different dimensions
 - General structure
 - Environment
 - Goals
 - Methods
 - User community
 - Licensing

The Third Design Space

- A metaphorical, not actual physical space where participants from corporations and open source communities come together to create a new design environment
- Participants create new design associations and circulate shared design resources that result in new shared software and innovative software development processes
- New software innovations not possible in a strictly commercial or an independent open source community are developed

Summary

- Systematic approach to identifying problems
- Systems analysts are required to take on many roles
- Analysts possess a wide range of skills
- The systems development life cycle (SDLC)
- Agile approach
- Object-oriented analysis and design
- Cloud computing
- Open source software
- Third Design Space

Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructors in teaching their courses and assessing student learning. Dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.